

第2回 AIエージェント社会シミュレーション・ハッカソン「メタ安全保障」

マルチASVスウォームの 攻防シミュレーション

複数の無人ボート(ASV)が2つの陣営に分かれ、"旗"を守る側と奪う側として戦う——
その一連の攻防を再現するシミュレーション基盤

github.com/sbilxxx/asv_swarm_dt / ブラウザで動く公開デモ (GitHub Pages) つき

守る側は"旗"を守り切れば勝ち、攻める側は旗に届けば勝ち

登場するもの

ASV=Autonomous Surface Vehicle。人が乗らず自分で動く無人ボートのこと。

守備側・攻撃側それぞれが複数隻のASVからなる艦隊(=スウォーム)を持ち、海域の中央に置かれた"旗"を挟んで対峙する。

勝敗のルール

- ・攻撃側が旗にたどり着けば**突破**(攻撃側の勝ち)
- ・制限時間内に守り切れば**防御成功**(守備側の勝ち)
- ・ボート同士が至近距離で出会うと**双方消える**(相討ち)

見る画面は2種類

同じ攻防を、3Dの海域(カメラ・レーダー視点)と2Dの俯瞰マップ(戦術図)の両方で再生できる。中身のルールはどちらも共通。

まずはこのゲームがどういうもののかの全体像。仕組みや理由は次のスライド以降で説明する。

「AIが安全保障の判断を担う社会」を、小さな海の攻防で試す

ハッカソンの主題

今回のテーマは「メタ安全保障」——AIエージェントが意思決定に関わる社会を、実際に動くシミュレーションで検証すること。

本作では、艦隊の指揮官やボート自身の判断をAIに任せたときに**何が起きるか**を実験できる場を作った。

使うAIの種類

LLM=文章で状況を読み、指示を返す会話AI。

VLM=カメラ画像を見て判断できるAI。

比較対象として、あらかじめ決めた手順で動く**ルールベース**(AIを使わない従来ロジック)も用意し、同じ土俵で比べる。

中心の問い

指揮官の判断が遅い、あるいは情報が古いと、戦況にどう影響するか。

この問いに答えられる基盤を作ることが今回のゴール。

共通のルールが1つ、それを見る画面が2つ

共通ロジック 動作

世界の状態・物理演算・センサー（GNSS／レーダー）・勝敗判定・記録ログを1か所にまとめている。

ブラウザでも、画面なしのサーバー実行（headless）でも、同じものがそのまま動く。

2つの画面 動作

3D海域ビュー — カメラ映像・レーダー画面を模したHUD付き。

2D俯瞰マップ — 攻防全体と各AIの判断口グを見渡せる戦術図。

どちらも同じ共通ロジックの上で、同じ攻防を別の角度から映すだけ。

設計の原則

- ・舞台となる海域は固定しない（設定ファイルで差し替え可能）
- ・サーバー不要・ビルド不要で、そのままウェブに公開できる
- ・2つの画面は完全に同じ座標系を使うため、見た目が違ってても矛盾しない

独自の仕掛け① — 見ている情報が違う

指揮官と現場のボートは、**違う情報を違うタイミング**で見て判断する



伝言ゲームのような構造

指揮官・現場のボート、どちらのAIも「状況を受け取り、命令を返す（または何もしない）」という同じ形の判断をしている。

形が同じだから、ルールベースとLLMを同じ土俵の上で差し替えて比較できる。

役割の違いはプロンプトで作らない

指揮官には**広いが古い**陣営全体の状況図を、現場のボートには**狭いが新しい**その場の観測結果を渡す。

「知っていることの食い違い」をあえて入力的设计で作ることで、指揮系統に意味が生まれる。

AIが返事を返すまでの間、船は**"古い命令"**のまま動き続ける

指示を出す
状況を送る瞬間



AIが考えている間
この間、船は前の命令のまま動く



指示が届く
新しい命令に切り替わる瞬間

なぜ重要か①: 公平に比較できる

この「考えている時間」は実際の計算時間の実測値ではなく、あらかじめ決めた設定値。

だから、ルールベースの指揮官とLLMの指揮官を同じ遅さの条件で比べられる。より速いパソコンに変えてもシミュレーションの結果は変わらない——変わるのは実行にかかる実時間だけ。

なぜ重要か②: 実験の軸にできる

「命令をどれだけ待つか」「間に合わなかったらどうするか」までが設定値なので、

「指揮が遅い陣営は負けるのか」「指揮系統が途切れたら何が起きるか」を、コードを変えずに設定を変えるだけで問える。

設計より実測が先、という家訓

1,554

回の実LLM推論(150エピソード連続)
ローカルLLM(Ollama・qwen2.5:7b)で
実行
解析失敗 1件／通信失敗 0件

108,600

1秒あたりのシミュレーション更新回数(3
隻・CPU1コア)
30隻でも6,400回/秒
headless実行で多数体×多エピソードに
拡張可能

VLM閉ループ

実際のレンダリング画像で成立(8/30)
3DCGの一人称視点画像 → VLMが判断
→ 航路を決定 → 船が動く、まで一通り繋がった
AI応答 中央値1.78秒／画像生成 中央
値84-115ミリ秒

指揮官をAIにしても、**従来ロジックに有意には勝てなかった**

測ったこと

3パターンを比較した: ①両陣営ともルールベース ②指揮官だけLLM ③両陣営ともLLM。各25エピソード。

同じ条件で繰り返し試行しても、勝率は約12ポイントの範囲でばらつく。LLM指揮官とルールベース指揮官の勝率の差はこのばらつきの範囲に収まってしまい、統計的に「差がある」とは言えなかった(有意差なし)。差を確かめるには1パターンあたり約170エピソードが必要と分かった。

診断 — なぜか

指揮の中身には確かに欠陥があった。**移動命令の80.9%が、直前と同じ座標への送り直し**——つまり行き先が更新されないまま同じ命令を出し続けていた。

だが根本原因はAIの弱さではなく——

盤面のルール側に、賢く指揮しても差が出る余地が無かった。

指揮を工夫しても結果が変わらないなら、それは指揮官の問題ではなくゲーム設計の問題である。

「どこまで近づいてよいか」が勝敗を分けるゲームへ

3種類のASVに役割を分けた。将棋の駒に例えると、下の対応になる。

艦種	将棋でいうと	速力	探知距離	爆破半径	役割
快速艇	歩	9 m/s	300 m	50 m	先鋒・護衛
索敵艇	金	6 m/s	1200 m	0 m 非武装	目(偵察役)
重装艇	飛車	4 m/s	400 m	100 m	本命(主力)

爆破半径 — 1つの数字が3つの意味を持つ

1. 対艦: この距離で相手を捉えれば、相討ちで双方が消える
2. 対旗: この距離まで旗に近づけば、旗を破壊できる
3. 巻き添え: 迎撃した位置がこの距離内なら、旗も一緒に壊れる

3があるから、守備側は旗の真上で待ち構えられない。必ず前に出て迎撃する必要がある。

中核にある不確実性

レーダーには「点」しか映らない。
それがどの艦種かは分からない。

本命(重装艇)がどれかは、近づいて識別するまで分からない——その識別を次段階でVLMに担わせる。

勝敗が散らばった。ゲームとして成立している

16

防御成功 80%

4

突破 20%

0

時間切れ
全20エピソードが決着

31.5秒

平均決着時刻
制限240秒に対して

決着時刻が2つの山に分かれた

防御成功は16~35秒に集中し、突破は75~81秒に集中した。その中間はほとんど無い。

「足の速い快速艇を序盤に止められたか」「足の遅い重装艇が生き延びて旗まで届いたか」で勝敗が分かれている——設計時に狙った2段階の時間割が、実際の盤面の上で効いている。

再現できること

両陣営ルールベースの統制群は完全に決定論的(乱数も実時刻も使わない)。誰がどの環境で実行しても、1バイト単位で同じ結果になる。

```
node scripts/headless_run.js
--scenario flag_defence_squadrons
--episodes 20
```

穴は「AIが1つも動いていない」から「主題であるAIの活躍がまだ足りない」へ

到達点

- 実測済 共通ロジック・攻防の勝敗判定・記録ログ
- 実測済 「考えている間も時間は進む」仕組み（発行→発効・締切・不成立）
- 実測済 指揮官LLM 1,554推論／3パターン比較
- 実測済 VLM閉ループ（実レンダリング画像・1隻分）
- 実測済 新盤面で決着（20エピソード・80%/20%・時間切れ0）
- 実装済 現場のボートAI（艇レベルLLMの骨格）

次にやること

- 次 識別台帳 — 近づいたら艦種が分かる仕組み
- 次 現場のボートAIの本格運用（主題の本体）
- 次 VLM索敵艇 — カメラ画像から本命（重装艇）を見抜く

計算資源が必要な理由：勝率の差を確かめるには1パターンあたり約170エピソードが要る。手元のGPU（RTX 3060）では6隻を回すのに処理能力が3.4倍足りないことを実測済み。